

System Architecture Document

MARGA: Quantum-Inspired Adaptive Traffic Routing Engine

SIH 2026 — Problem Statement 26137 | Egreen Quanta | Architecture v2.0, Hackathon Build

1. Recommended Tech Stack

Every choice below is deliberately the simplest tool that can do the job, not the most impressive one. A judge or a future engineer reading this should be able to see why each piece is there.

Layer	Technology	Why This, Not Something Heavier
Frontend framework	React (Vite) + TypeScript	A single-screen demo dashboard doesn't need server-side rendering, so plain Vite React avoids the extra build/deploy layer a framework like Next.js would add for no real benefit here.
UI styling	Tailwind CSS	Utility classes let a small team style quickly without a separate design system.
Map	MapLibre GL JS	Free and open-source, no API key or billing account required, unlike Google Maps JS.
Charts	Recharts	Lightweight React charting for benchmark and convergence graphs.
State management	Plain React state (useState/useContext)	At this data volume, a dedicated state library (Zustand) or query-cache layer (TanStack Query) solves a scaling problem the project does not have.
Backend	Python + FastAPI	Matches the team's existing experience; Python also hosts every optimization library needed below.
Optimization core	NumPy/SciPy — custom QPSO / VA-QPSO	The team's own algorithm, written from scratch since it is the core contribution.
Classical baseline	Google OR-Tools	Free, well-proven VRP solver used for honest benchmarking.
Road graph	OSMnx + NetworkX	Turns real OpenStreetMap data into a weighted graph in a few lines of Python.
Traffic model	Simulated / synthetic time-varying weights (SUMO optional)	Reproducible and reliable for a live demo; no dependency on an unpredictable live feed.
Database	PostgreSQL	Relational structure fits the data (see Section 4); Railway provides it free.
Deployment	Docker + Vercel (frontend) + Railway (backend & database)	Same pattern the team has already used successfully before.

Kept deliberately out of the core stack: deck.gl (a WebGL animation library). It produces impressive vehicle animation, but is heavy to learn under a deadline. A simple marker animated along the route

polyline achieves most of the visual effect for a fraction of the engineering time, and remains a fine stretch goal if time allows.

2. Frontend Architecture

The frontend only visualizes and collects input. It never runs optimization itself — all computation happens on the backend, which keeps the frontend simple and keeps the one performance-critical piece (the solver) in one place.

- **Live Route Map** — the road network, vehicles, stops, and computed routes, rendered with MapLibre; lightweight markers move along route polylines using route timestamps.
- **Optimization Controls** — scenario and solver selection, and the “Optimize” action.
- **Incident Control** — click a road segment to simulate a closure or congestion spike — turns the demo into something the judges can interact with, not just watch.
- **Volatility / β Inspector** — shows each zone's measured traffic volatility and the resulting β value, making VA-QPSO's core idea visible instead of hidden inside the algorithm.
- **Benchmark Panel** — cost, runtime, and convergence comparison between VA-QPSO and OR-Tools, with optional GA/ACO baselines if time remains.

Impact Metrics Panel — translates results into time, distance, fuel, and CO2 savings using clearly documented assumptions, so estimates are not mistaken for measured data.

3. Backend Architecture

- **Graph Service** — loads and caches the OSMnx road graph so the demo never depends on a live download.
- **Optimization Engine** — runs VA-QPSO and OR-Tools behind one common solver interface, so optional GA/ACO baselines can be added without touching the API layer.
- **Encoding & Constraints Module** — converts QPSO's continuous particle values into a valid discrete route (random-key encoding) and repairs/penalizes capacity or time-window violations.
- **Traffic Engine** — maintains two separate states — background/external traffic (drives volatility and congestion detection) and controlled-fleet state (MARGA's own vehicles). These are kept apart on purpose (see Section 4 and Section 11) so the platform's own vehicles never inflate the congestion signal they are reacting to.
- **Re-optimization Engine** — checks a configurable disruption threshold with a cooldown rule; only when it is crossed does it identify the affected zone/vehicles and re-solve that subset, leaving unaffected routes untouched.
- **Streaming & Benchmark Engine** — exposes GET /stream/{run_id} via SSE for vehicle positions, beta, and volatility during a run, and computes CVRPLIB/Solomon comparisons.

4. Database Choice and Data Relations

PostgreSQL is the right fit here because the data is small in volume but rich in relationships — a scenario has stops, a run belongs to a scenario, a route has an ordered list of stops — which is exactly what a relational database is built for. A document database would just mean rebuilding these joins by hand in application code.

Entity	Stores	Relationship
Scenario	City area, fleet size, vehicle capacity, stop list, configuration	One scenario has many optimization runs
OptimizationRun	Solver used, total cost, runtime, random seed, timestamp	Belongs to one scenario; has many routes
Route	Vehicle id, ordered stop sequence, path, timestamps, distance, load	Belongs to one run
TrafficZoneState	Zone id, timestamp, volatility score, resulting β , source flag (background vs. controlled fleet)	Independent log table; read by the Re-optimization Engine, never written to by the fleet's own vehicle data

The separate TrafficZoneState table, with an explicit source flag, is the concrete database-level enforcement of the PRD's requirement that MARGA's own vehicles must never be counted as the traffic input that triggers rerouting.

5. Authentication Approach

None, by design. The PRD marks accounts and multi-tenancy as out of scope, and there is no real multi-user need in a hackathon demo. If the event network needs any protection at all, a single shared API key checked in request headers is sufficient — a few lines, not a real authentication system.

6. External APIs and Services

- OpenStreetMap, via OSMnx — free, no API key, used once to build the road graph.
- CVRPLIB and Solomon benchmark files — static datasets downloaded once, not a live API.
- No live paid traffic API (HERE/TomTom) — deliberately out of scope per the PRD; simulated/background traffic is used instead, which is free and fully reproducible for a live demo.

7. AI Model Integration

No generative AI or LLM is required, and adding one would dilute the pitch — PS 26137 asks for an optimization algorithm, not a language model. The “intelligence” in this system is VA-QPSO and OR-Tools, which is a stronger and rarer technical story at a hackathon than another LLM wrapper. A lightweight traffic-volume prediction model (simple regression, not an LLM) is a legitimate future extension, but is not part of the MVP.

8. Complete Request and Data Flow

1. Dashboard loads -> GET /graph -> cached road network rendered on the map
2. User defines a scenario (or picks a preset) -> POST /optimize
3. Backend runs VA-QPSO and OR-Tools on the same problem
4. Scenario, OptimizationRun, and Route rows are saved to PostgreSQL
5. Both routes + cost/runtime/impact comparison returned -> map and chart update; route timestamps support marker animation / 6. GET /stream/{run_id} sends SSE vehicle positions, beta, and volatility to the dashboard during the run
7. Background traffic simulator updates zone conditions independently

- of any MARGA fleet vehicle (this is the isolation from Section 4)
8. Judge clicks Incident Control (or the background simulator crosses the configured threshold) -> POST /reoptimize
 9. Backend checks threshold + cooldown -> if genuinely disruptive, identifies only the affected zone/vehicles -> re-solves that subset -> unaffected routes remain untouched
 10. Updated affected route(s) returned -> only those routes redraw on the map, with a before/after metric shown
 11. GET /benchmark independently returns CVRPLIB/Solomon comparison results for the Benchmark Panel

Note on the incident trigger: the PRD's threshold-and-cooldown logic runs identically whether the disruption comes from the live background simulator or a judge manually clicking “simulate incident.” Wiring the demo to the manual trigger as the primary path (Section 13) removes any risk of the disruption not happening at the right moment on stage, without changing or weakening the underlying engineering.

9. Folder Structure

```

marga/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── MapView.tsx
│   │   │   ├── ControlPanel.tsx
│   │   │   ├── IncidentControl.tsx
│   │   │   ├── VolatilityInspector.tsx
│   │   │   └── BenchmarkPanel.tsx
│   │   ├── api/                (fetch calls to backend)
│   │   └── types/
│   └── package.json
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   │   ├── graph.py
│   │   │   ├── optimize.py
│   │   │   ├── reoptimize.py
│   │   │   └── benchmark.py
│   │   ├── solvers/
│   │   │   ├── base.py
│   │   │   ├── qpso.py          (VA-QPSO)
│   │   │   └── or_tools_solver.py
│   │   ├── graph/              (OSMnx loading + caching)
│   │   ├── traffic/            (background traffic + fleet state, kept separate)
│   │   ├── routing/            (encoding, constraints, re-optimization)
│   │   ├── benchmark/          (CVRPLIB/Solomon runner)
│   │   ├── models.py           (SQLAlchemy tables)
│   │   └── schemas.py          (Pydantic request/response shapes)
│   ├── benchmark_data/
│   └── requirements.txt
├── docker-compose.yml
└── README.md

```

10. Major Components

- Graph Service — loads and caches the road network
- Solver Layer — VA-QPSO and OR-Tools behind one common interface
- Encoding & Constraint Module — the core hard-engineering piece: continuous particle to valid discrete route
- Traffic Engine — background traffic and controlled-fleet state, kept isolated from each other
- Re-optimization Engine — threshold and cooldown logic, affected-zone detection, partial re-solve
- Streaming & Benchmark Engine — SSE live positions, beta/volatility, and standard-dataset metrics
- API Layer — FastAPI endpoints tying everything together
- Database — PostgreSQL, storing scenarios, runs, routes, and traffic-zone logs
- Dashboard — map with timestamped marker animation, controls, volatility inspector, impact metrics, and benchmark charts

11. Security and Reliability Considerations

- Validate every incoming request with Pydantic models — malformed data is rejected before it reaches the solver.
- Cap problem size (for example, a 500-stop limit) on /optimize so no request can freeze the demo server.
- Keep PostgreSQL private; only the FastAPI service talks to it directly.
- Use synthetic/demo data only — there is no real personal data in this system, which sidesteps most privacy concerns entirely.
- Keep background-traffic state and controlled-fleet state in separate tables/services, per Section 4 — this is a correctness control as much as a security one, directly addressing the feedback-loop risk named in the PRD.
- Give the frontend an offline/mock-data fallback so a backend hiccup during the demo does not take down the whole presentation.
- Cache the road graph ahead of time rather than depending on a live OpenStreetMap download during the demo.

12. Deployment Architecture

```
Browser
  |
  v
Vercel (React frontend)
  | HTTP
  v
Railway (FastAPI backend, Dockerized)
  |
  v
Railway (managed PostgreSQL)
```

Frontend and backend are deployed separately so that a backend issue does not take the entire dashboard offline — the frontend's mock-data fallback (Section 11) covers exactly this case.

13. Hackathon Simplifications

- Use one locked Bengaluru subgraph for the main demonstration, not a free-form area picker.
 - Use simulated/synthetic traffic first; attempt SUMO only if it does not delay the core optimizer.
 - Make OR-Tools the single primary baseline; keep the interface ready for optional GA/ACO only if time genuinely remains.
 - Use two or three preset scenarios instead of building a full scenario editor.
 - Trigger the demo's traffic disruption manually (Incident Control button) as the primary path, not a live autonomous timer, so the moment is 100% reliable on stage — the same threshold/cooldown code runs either way.
 - Do not build authentication, user accounts, a mobile layout, browser-side optimization, or a settings system.
 - Record a full end-to-end run before demo day as a backup in case of live technical issues.
-

Architecture Principle

Keep the system simple enough for a small team to build and explain, but rigorous enough to defend under a judge's questions. Every component above exists because it maps directly to a line in the PRD — graph-based routing, quantum-inspired optimization, honest benchmarking, and traffic-aware rerouting that is stable rather than reactive to every fluctuation. Nothing is included because it sounds impressive.